Chapter Two

Literature Review

2.1 Introduction

This chapter reviews the body of published work that bears on the design and implementation of a multi-tenant SaaS inventory management platform. The purpose of the review is threefold. First, it establishes the conceptual foundations of the three areas on which the project rests: the Software-as-a-Service delivery model, multi-tenant architecture, and microservices. Second, it examines the empirical literature industrial experience reports, case studies, and implemented systems to determine which design approaches have been shown to work in practice and where their limits lie. Third, it situates the project within the Nigerian small business environment, where the platform is intended to be used, and identifies the specific gap in the existing literature and in existing systems that this project addresses.

The review is organised as follows. Section 2.2 presents a conceptual review of the key ideas: cloud computing and SaaS, multi-tenancy, multi-tenant data architecture, microservices, and inventory management systems. Section 2.3 discusses the theoretical framework within which the design decisions of this project are made, drawing on service-oriented architecture theory and the SaaS maturity model. Section 2.4 reviews the empirical studies that have examined multi-tenancy and microservices in real systems. Section 2.5 reviews existing inventory management systems, both commercial platforms and systems developed by researchers in Nigeria. Section 2.6 examines the literature on technology adoption among Nigerian SMEs. Section 2.7 summarises the reviewed studies in tabular form. Section 2.8 draws the threads together to state the gap this project fills, and Section 2.9 concludes the chapter.

2.2 Conceptual Review

2.2.1 Cloud Computing and the Software-as-a-Service Delivery Model

Cloud computing is the enabling infrastructure for everything discussed in this project. Armbrust et al. (2010) offered what has become a standard working definition, describing cloud computing

as both the applications delivered as services over the internet and the hardware and systems software in the data centres that provide those services. Their analysis emphasised three characteristics that distinguish cloud computing from earlier hosting arrangements: the appearance of infinite computing resources available on demand, the elimination of any up-front commitment by the user, and the ability to pay for computing resources on a short-term basis as they are needed. These characteristics matter to this project because they shift the economics of software delivery. A vendor no longer needs to purchase hardware sized for peak demand before acquiring its first customer, and a customer no longer needs to buy hardware at all.

Within cloud computing, the literature conventionally distinguishes three service models: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). Kavis (2014) explains the distinction in terms of how much of the technology stack each party manages. Under IaaS, the provider supplies raw computing, storage, and networking, while the customer manages everything above the operating system. Under PaaS, the provider also manages the runtime and middleware on which the customer's application is built. Under SaaS, the provider manages the entire stack, and the customer simply uses the finished application, typically through a web browser. The platform developed in this project is a SaaS offering: the inventory application, its underlying services, and its data stores are all operated by the vendor, and tenant businesses interact with it only as users.

Kavis (2014) is careful to draw a line between a true SaaS application and a traditional application that merely happens to be hosted on someone else's server. The distinguishing characteristics he identifies include subscription-based pricing, centralised management and updating by the provider, and most importantly for this project a multi-tenant architecture in which a single instance of the software serves many customers. This distinction runs through the whole of the literature reviewed in this chapter: simply moving a single-tenant application into the cloud does not deliver the cost benefits that make SaaS attractive, because the provider still carries a full unit of infrastructure for every customer.

Chong and Carraro (2006), writing from an industry perspective at Microsoft, approached the same point from the vendor's side. They argued that SaaS makes it economically possible to serve the "long tail" of small businesses customers who could never justify the cost of licensed, on-premise software precisely because sharing infrastructure across many tenants drives the cost

of serving each additional tenant down towards zero. The business owner running a single shop in Osogbo is exactly the sort of customer their model describes: commercially unreachable under the old delivery model, but reachable under SaaS if and only if the vendor's architecture keeps per-tenant costs low. That condition is what leads directly to the concept of multi-tenancy.

2.2.2 The Concept of Multi-Tenancy

Multi-tenancy is the architectural property that allows one running system to serve many customer organisations at once. Although the term is used constantly in both industry and academia, Kabbedijk et al. (2015) found that it had no settled definition. In a systematic mapping study covering 761 academic papers and 371 industrial blog posts, they found that definitions varied considerably even within each community. From that analysis they synthesised a definition that has been widely adopted: multi-tenancy is "a property of a system where multiple customers, so-called tenants, transparently share the system's resources, such as services, applications, databases, or hardware, intending to lower costs, while still being able to exclusively configure the system to the needs of the tenant" (Kabbedijk et al., 2015). Three elements of this definition deserve attention in the context of this project. First, the sharing is transparent a tenant should not be aware that others are using the same system. Second, the purpose is cost reduction, which is the whole economic argument for SaaS in price-sensitive markets. Third, sharing does not mean uniformity: each tenant retains the ability to configure the system as if it were their own.

Kabbedijk et al. (2015) also clarify two distinctions that are easy to blur. A multi-tenant system is not simply a multi-user system: a multi-user application such as a social network offers every user the same functionality, whereas a multi-tenant application allows each customer organisation substantial scope to configure its own environment. Nor is multi-tenancy the same as multi-instance deployment, where the provider spins up a fresh copy of the application for each customer that arrangement shares nothing and therefore delivers none of the economies of scale.

Guo et al. (2007) approached multi-tenancy from the engineering side, asking what it takes to build an application that is multi-tenant "natively" that is, designed for sharing from the outset rather than converted later. They identified a set of requirements that a native multi-tenant

framework must satisfy, including tenant-specific customisation, security isolation between tenants, performance isolation so that one tenant's workload cannot degrade the service experienced by others, and the ability to scale as the tenant population grows. They also drew the explicit connection, echoed by Kabbedijk et al. (2015), between multi-tenancy and economies of scale: sharing is what allows a provider's profit margin to grow as the customer base grows.

The benefits come with well-documented engineering challenges. Bezemer et al. (2010), reporting on the conversion of a real single-tenant industrial application to multi-tenant operation, identified the recurring problem areas as tenant identification making sure every request and every record is correctly associated with the tenant it belongs to data isolation, and performance management. Their report is sobering reading for anyone planning such a system: the conversion touched a large fraction of the existing codebase, and they identified recurring code-level patterns that reliably cause trouble, such as global caches and static variables that silently leak data across tenant boundaries. The lesson this project takes from their work is that tenant-awareness cannot be bolted on afterwards; it has to be present in every layer of the design, from the authentication token down to the structure of the database.

2.2.3 Multi-Tenant Data Architecture and Isolation Strategies

Of all the design decisions in a multi-tenant system, the one with the most far-reaching consequences is how tenant data is stored. Chong, Carraro and Wolter (2006) described three approaches that have since become the standard taxonomy in the literature, arranged along a continuum from fully isolated to fully shared.

The first approach is the separate database model: each tenant gets its own database instance. This provides the strongest possible isolation a query error or a security breach in one tenant's environment cannot physically touch another tenant's data and it makes per-tenant backup, restore, and migration straightforward. Its weakness is cost. Database servers are expensive to run, and there is a practical limit to how many databases one server can host. A provider serving hundreds of small tenants would see its infrastructure costs climb in direct proportion to its customer base, which undermines the entire economic rationale of SaaS for low-income markets.

At the opposite extreme is the shared database, shared schema model: all tenants' records sit in the same tables, distinguished only by a tenant identifier column on each row. This is the

cheapest model to operate and the easiest to scale to very large tenant populations, but it places the entire burden of isolation on the application layer. Every query must include the correct tenant filter, and a single missing filter in a single code path exposes every tenant's data. Backup and restore for an individual tenant also become complicated, because that tenant's rows are intermingled with everyone else's.

The third approach sits between the two: a shared database with a separate schema for each tenant. All tenants share one database server, but each tenant's tables live in their own schema, their own namespace within it. Isolation is enforced by the database engine's own access controls rather than by application-level filtering alone, which removes the most dangerous failure mode of the shared schema model, while the cost per tenant remains far below that of running separate database servers.

The research literature has generally treated these approaches as a trade-off to be navigated rather than a problem with a single right answer. Aulbach et al. (2008) studied the shared-schema end of the spectrum in depth, examining techniques for mapping many tenants' logically distinct schemas onto one shared physical schema. They showed that naive mapping techniques such as universal tables in which any attribute can go into any generic column can degrade query performance badly, and they proposed a technique called chunk folding that preserves performance by partitioning logical tables into chunks matched to physical tables of a similar structure. Their work demonstrates what can be achieved at the extreme shared end of the continuum, but it also illustrates the complexity that extreme sharing drags in: their techniques were developed in the context of enterprise database research and require careful tuning that would be out of proportion for a small platform serving small businesses.

For this project, the shared-database, separate-schema model is the appropriate point on the continuum, and the literature supports that judgement. It offers isolation strong enough to be defensible to tenants, at a cost structure compatible with the Nigerian SME market, and it does so without the engineering complexity documented by Aulbach et al. (2008). The performance and isolation characteristics of this choice are evaluated empirically in Chapter Four.

2.2.4 Microservices Architecture

The second architectural pillar of this project is microservices. Dragoni et al. (2017) trace the style's intellectual lineage from object-oriented programming through component-based development and service-oriented architecture (SOA), presenting microservices as the most recent step in a long movement towards smaller, more loosely coupled units of software. Their survey is candid about the field's immaturity at the time of writing: there was no single agreed definition, and many of the claimed benefits rested on industrial experience rather than rigorous evaluation.

The definition most commonly used in practice is close to Newman's. Newman (2021) describes microservices as small, independently deployable services modelled around a business domain, communicating with each other over a network through technology-agnostic protocols. Two ideas in that description carry most of the weight. The first is independent deployability: a service can be changed and released without requiring any other service to change or redeploy. The second is domain modelling: service boundaries should follow the boundaries of the business inventory, orders, and notifications rather than technical layers such as presentation, logic, and data access. A monolithic application, by contrast, couples all of its functionality into a single deployable unit, so that releasing a change to one part means rebuilding and redeploying the whole.

The benefits claimed for microservices in the literature are significant but not free. On the positive side, independent deployability reduces the risk and downtime associated with releases; services can be scaled individually, so that a platform can add capacity to its busiest function order processing during a sales peak, say without paying to scale the rest; and teams can work on different services concurrently without constantly interfering with each other (Newman, 2021; Dragoni et al., 2017). On the negative side, a microservices system is a distributed system, and distribution brings its own costs: network calls replace in-process calls, data consistency must be managed across service boundaries, and operational complexity increases because there are simply more moving parts to deploy, monitor, and debug. Newman (2021) is explicit that microservices are not a default choice, and that the decision to adopt them should rest on a genuine need for the properties they provide.

This project has such a need, and the literature bears directly on why. In a multi-tenant platform, an update that requires restarting the entire application interrupts service for every tenant at once every shop using the platform for its day's trading. Batista et al. (2024), studying asynchronous workloads in a multi-tenant microservice environment, documented how independently deployable services, coordinated through message queues rather than direct calls, allow workloads from many tenants to be processed without one tenant's surge overwhelming shared components. The combination of microservices with asynchronous messaging is what makes it practical to fix or scale the notification service, for example, while the inventory and order services continue trading normally.

2.2.5 Inventory Management and Computerised Inventory Systems

The application domain of this project is inventory management: the set of practices by which a business tracks the goods it holds, controls when and how much it reorders, and keeps the cost of holding stock in balance with the risk of running out. The core questions what do we have, what do we need, and when must we reorder predate computing entirely, and Chapter One traced the discipline's computerisation from the Material Requirements Planning systems of the late 1960s through to today's enterprise resource planning platforms.

What matters for this review is the state of computerised inventory management at the small-business end of the market, because that is the population this project serves. The literature on small-firm inventory systems is thinner than the literature on enterprise systems, but a consistent picture emerges from it. Small businesses that computerise their stock records gain better visibility of stock levels, fewer discrepancies between recorded and physical stock, and more reliable reorder decisions; those that remain on manual or spreadsheet-based records carry all of these problems in reverse. Erameh and Odoh (2021), introducing their web-based inventory control system for a Nigerian SME, observed that inventory data in most small businesses is recorded manually into spreadsheets and maintained with little or no technological innovation, making it genuinely difficult to monitor inventory reliably. Chukwumuanya, Onwurah and Ihueze (2024) make a closely related observation about Nigerian small-scale manufacturing: poor record-keeping of what is procured, used, and sold leaves managers making decisions on faulty data.

A further point concerns the shape that small-business inventory software tends to take. The implemented systems reported in the literature, including the two Nigerian systems just cited, reviewed in detail in Section 2.5 are almost uniformly single-organisation applications: built for one business, installed for one business, and holding one business's data. That is a natural way to build such a system, but it means each deployment carries its own full cost of hosting and maintenance. The commercial SaaS platforms reviewed in Section 2.5.1 show the alternative, multi-tenant approach at work but at price points, and with workflow assumptions, aimed at markets very different from Nigeria's. The intersection an inventory platform that is both genuinely multi-tenant and aimed at the economics of a Nigerian small business is precisely where the literature is silent, as Section 2.8 sets out.

2.3 Theoretical Framework

2.3.1 Service-Oriented Architecture

The theoretical root of the microservices architecture used in this project is service-oriented architecture. Dragoni et al. (2017) describe the historical progression plainly: software architecture moved from monoliths to objects, from objects to components, from components to services, and from coarse-grained services to microservices. Each step reduced the size and coupling of the units from which systems are assembled, and each step was driven by the same underlying motivation the desire to change one part of a system without having to understand, rebuild, and redeploy all of it.

Service-oriented architecture formalised several ideas that this project inherits. Services expose their functionality through well-defined, documented interfaces and hide their internal implementation behind those interfaces. Services are autonomous: each one can be developed, deployed, and operated on its own schedule. And services communicate through standard, technology-neutral protocols, so that the choice of implementation technology inside one service does not constrain the choices made in another. Microservices, in Newman's (2021) account, are essentially service-oriented architecture practised with much smaller services and with modern tooling: containers, automated deployment pipelines, and lightweight HTTP-based communication that makes the fine-grained approach practical where it once was not.

This theoretical grounding shapes two concrete design decisions in the present project. The first is the drawing of service boundaries along business-domain lines: authentication and tenant management, inventory, orders and sales, notifications rather than along technical layers, following the principle that a service should encapsulate a coherent area of business capability (Newman, 2021). The second is the enforcement of loose coupling: services never call into each other's databases, and cross-service communication happens either through the REST APIs exposed at each service's boundary or asynchronously through the message broker. Both decisions are direct applications of service-orientation principles documented in the literature, and both are evaluated in Chapter Four against the project's stated objectives.

2.3.2 The SaaS Maturity Model

The second element of the theoretical framework is the SaaS maturity model proposed by Chong and Carraro (2006), which provides a graded scale against which any SaaS architecture, including the one built in this project, can be located. The model defines four levels of maturity.

At Level I, termed ad hoc or custom, each customer runs its own customised instance of the hosted application. Architecturally, this is barely distinguishable from traditional on-premise software, and it captures few of SaaS's economic benefits, since the provider must operate and maintain as many distinct versions of the application as it has customers. At Level II, the provider still hosts a separate instance per customer, but all instances run identical code; differences between customers are handled through configuration rather than customisation. This reduces maintenance effort considerably, because an update to the shared codebase reaches all customers at once, but the provider still carries the infrastructure cost of one instance per tenant. At Level III, a single application instance serves every customer, with configuration metadata providing each tenant its own behaviour and with authorisation policies keeping each tenant's data separate. This is where the real economies of scale begin, but Chong and Carraro (2006) note a limitation: a single instance can only be scaled up to a more powerful server, and that route has diminishing returns. Level IV resolves the limitation by running the multi-tenant application across a load-balanced farm of identical instances, so that capacity is added by adding servers rather than by enlarging one.

The maturity model does double duty in this project. First, it provides a framework for discussing the platform's multi-tenant design and possible scaling path. The platform uses a shared application codebase and a shared PostgreSQL database with a separate schema for each tenant. Its microservices structure allows individual services to be considered for independent replication as demand increases. However, Level IV maturity requires deployment of replicated application or service instances with suitable load balancing. Since the core implementation and evaluation in this project do not establish a load-balanced farm of replicated instances, the study does not claim to have achieved Level IV. Rather, it adopts a design that can support future horizontal scaling when the necessary deployment infrastructure is available. Second, the model provides a vocabulary for explaining why single-tenant inventory tools common in the Nigerian market fail to achieve the shared infrastructure economics associated with multi-tenant SaaS (Chong & Carraro, 2006; Kavis, 2014).

Together, the two elements of the theoretical framework service orientation as the basis for the system's decomposition, and the SaaS maturity model as the basis for its tenancy and scaling design define the architectural position this project takes. The empirical literature reviewed in the next section examines how these ideas have performed when applied to real systems.

2.4 Empirical Review

2.4.1 Multi-Tenancy Design and Enablement Studies

Guo et al. (2007) presented one of the earliest structured treatments of what they called native multi-tenancy applications conceived for shared, multi-customer operation from the start rather than adapted to it. Working from an analysis of the requirements such applications face, they proposed a development and management framework covering tenant identification, security isolation, customisation, and quality-of-service management. The study is valuable to this project less for its specific framework than for its enumeration of the requirements themselves: every one of the requirements they identified in 2007 knowing which tenant a request belongs to, keeping tenants' data and configuration apart, and preventing one tenant's load from harming another's experience reappears as a design obligation in the present system. The work is conceptual rather than evaluative, however; the framework was not validated against a large

deployed system, and the authors acknowledged that implementation-level questions remained open.

Mietzner et al. (2009) addressed a problem that any SaaS provider serving varied customers will meet: how to let different tenants configure the application differently without forking the codebase. They applied variability modelling a technique drawn from software product line engineering, in which the points at which a system can vary are made explicit as a model to the customisation and deployment of multi-tenant SaaS applications. Their results showed that a variability model can drive automated, tenant-specific deployment, reducing the effort of configuring each new tenant's variant. The work is directly relevant to the onboarding problem in this project: when a new Nigerian business registers on the platform, its environment must be provisioned without manual engineering effort. The limitation of the study is that it remained largely at the level of models and prototypes; it did not measure the approach under the sustained load of a large tenant population.

Bezemer et al. (2010) supplied the study that comes closest to a cautionary tale. They documented, at source-code level, the conversion of an existing single-tenant industrial application into a multi-tenant one. The exercise surfaced recurring patterns that obstruct multi-tenant operation among them globally shared caches, static state, and hard-coded assumptions that one installation equals one customer. Their experience report established two things this project relies on. The first is that tenant identification and data isolation must be designed into every layer, because retrofitting them is disproportionately expensive. The second is that performance management in a multi-tenant system is a distinct engineering problem in its own right: the provider must now reason about fairness between tenants, not merely about the responsiveness of the system as a whole. Because the study reports a single conversion of a single legacy codebase, its findings cannot be assumed to generalise to greenfield designs, but for a project building a multi-tenant system from scratch, its central lesson is liberating rather than constraining: the problems it catalogues are largely avoidable when tenant-awareness is present from the first design decision.

2.4.2 Multi-Tenant Data Management Studies

The data layer has attracted its own strand of empirical work. Aulbach et al. (2008) examined the database problem at the shared end of the tenancy continuum, where many tenants' logically distinct schemas must be mapped onto one physical schema. They demonstrated that the straightforward mapping strategies in common use universal tables and pivot tables, in which tenant-specific attributes are squeezed into generic column structures carry serious performance penalties, and they proposed and evaluated an alternative, chunk folding, in which logical tables are vertically partitioned into chunks that are folded into physical multi-tenant tables matched to their structure. Their experimental results showed chunk folding retaining good performance where the generic approaches degraded. Two implications matter for this project. First, the extreme consolidation end of the continuum is workable but technically demanding; the sophistication of their solution is itself evidence of how much complexity extreme sharing creates. Second, and conversely, a project that needs solid isolation and moderate scale rather than maximal consolidation is better served by the middle option, the separate-schema model which achieves its isolation through ordinary database mechanisms rather than through elaborate mapping machinery. That is the position this project takes, and the reasoning is documented further in Chapter Three.

The practitioner literature aligns with this reading. Chong, Carraro and Wolter (2006) analysed the three data architecture options separate databases, shared database with separate schemas, and shared database with shared schema explicitly as a trade-off between isolation and economy, and recommended that providers choose based on the isolation expectations of their target market and the cost structure of their infrastructure. For a platform aimed at Nigerian SMEs, where tenants are entitled to expect that a competing business on the same platform cannot read their stock figures, but where subscription prices must remain low, the separate-schema option is the natural compromise, and this project adopts it.

2.4.3 Microservices and Multi-Tenancy in Industrial Settings

A small but important group of studies has examined microservices and multi-tenancy operating together in real industrial systems, and these come closest to the architecture of the present project.

Tizzei et al. (2017) studied a multi-tenant SaaS product built on microservices in an industrial setting, combining the microservices style with software product line engineering to manage the variation between tenant deployments. Tracking the system's evolution over time, they measured an average of 62 percent of lines of code reused across tenant deployments concrete evidence that a microservices-based multi-tenant system can absorb tenant-specific variation without the codebase splintering into per-customer versions. The finding is encouraging for this project, whose tenants will differ in their product types, units of measure, and workflow details. The study's limitation, acknowledged in the proposal stage of this project, is that its evidence comes from one industrial system; how well the result transfers to other domains remains an open empirical question.

Batista et al. (2022) reported the migration of a legacy monolith to a multi-tenant microservice architecture in an enterprise context. Their experience was positive on the points that matter here: the migration improved the system's scalability, eased its management, and made integration with third-party systems smoother, because each integration could be handled by a focused service rather than by changes threaded through a monolith. As with Tizzei et al. (2017), the evidence base is a single company's migration, and the authors' evaluation was confined to that context but the direction of the result is consistent with the broader literature on microservices (Newman, 2021; Dragoni et al., 2017).

Batista et al. (2024) extended this line of work to a question that turns out to be central in multi-tenant operation: how to manage asynchronous workloads, background jobs, notifications, and scheduled processing when many tenants share the same services. Their study of workload management in a multi-tenant microservice enterprise environment showed that routing cross-service work through message queues, with explicit attention to fairness between tenants, allows heavy background load from one tenant to be absorbed without degrading the responsiveness of the platform for others. This finding directly shaped the present project's decision to connect its services through a message broker rather than through synchronous calls alone: low-stock alerts and notifications are exactly the kind of asynchronous, per-tenant workload their architecture is designed to absorb.

Taken together, the empirical studies support three conclusions. Multi-tenancy is well understood at the level of requirements and patterns, and the main failure modes occur when

tenant-awareness is retrofitted rather than designed in (Bezemer et al., 2010; Guo et al., 2007). The data layer offers a genuine and well-documented choice between isolation and economy, with the separate-schema model occupying the defensible middle (Chong et al., 2006; Aulbach et al., 2008). And the combination of microservices with multi-tenancy has been shown, in industrial settings, to deliver reuse, scalability, and manageable per-tenant variation (Tizzei et al., 2017; Batista et al., 2022; Batista et al., 2024). What the existing studies share is a setting: large, well-resourced organisations in developed markets. None of them asks the question this project asks whether the same architecture can be made to work at the cost level and within the infrastructure realities of Nigerian small businesses.

2.5 Review of Related Systems

2.5.1 Commercial Inventory Management Platforms

The commercial market for inventory software is mature and crowded, and a small number of platforms dominate the segment reachable by small businesses. Zoho Inventory provides stock tracking, order management, and integration with online sales channels on a subscription basis. Odoo offers inventory as one module of a broader open-source business suite. QuickBooks Commerce, formerly TradeGecko, built its reputation on inventory and order management for small wholesalers and was absorbed into Intuit's product line. At the more demanding end, platforms such as Cin7 and Linnworks serve retailers and distributors operating across many sales channels and warehouses.

These products demonstrate that the SaaS model works for inventory management: they serve large customer bases from shared infrastructure, they are accessed entirely through the browser, and their continued commercial success confirms that small businesses will adopt hosted inventory tools when the tools fit their means. Their limitations, from the standpoint of this project, are not technical but economic and contextual. As Chapter One detailed, their subscription pricing is calibrated to European and North American markets and converts to sums that are difficult for a small Nigerian business to justify; their workflows assume payment infrastructure, shipping integrations, and tax regimes that do not match Nigerian practice; and their support operations are located in distant time zones. The commercial literature on these platforms is marketing material rather than engineering documentation, so their internal

architectures are not published but their pricing pages alone are sufficient to establish the affordability gap that motivates this project.

2.5.2 Locally Developed Inventory Systems

The research literature contains two recent and relevant implementations of inventory systems built in Nigerian universities, and they repay close attention because they show both how far local implementations have come and what they leave unaddressed.

Erameh and Odoh (2021) designed and implemented a web-based inventory control system for a small and medium enterprise, using the MERN stack MongoDB, Express, React, and Node.js under a waterfall development process. Their system supported distinct user roles (administrator, store manager, and sales representative), stock tracking with barcode and QR-code identification, sales recording, and report generation, and it was designed to give a corporate office daily, accurate visibility of inventory levels across its stores. The study demonstrates that a web-based system built with mainstream open technologies can meet the core inventory needs of a Nigerian SME. Its architecture, however, is a single-organisation monolith: one business, one deployment, one database. Extending it to serve a second business would mean standing up a second copy of everything, which reproduces exactly the per-customer cost problem analysed in Sections 2.2.3 and 2.3.2. Their evaluation, moreover, was functional; the system was demonstrated to work rather than a measurement of performance under load or of isolation between tenants, since there were no other tenants to isolate from.

Chukwumuanya, Onwurah and Ihueze (2024) developed a web-based inventory management system for small businesses, applied to a plastics production unit at Nnamdi Azikiwe University. Their contribution went beyond record-keeping: they fitted an ARMA(1,1) time-series model to the unit's historical production data and embedded the resulting demand forecast, together with economic order quantity calculations, in the system, which was built with a Python back end and a MySQL database and deployed to a cloud server. User testing found the system a marked improvement over the unit's previous manual practice. The study is important to this project for two reasons. It confirms that demand forecasting identified in Chapter One as beyond this project's scope can be added to a small-business inventory system, which makes it a documented direction for the platform's future development. And it shares the architectural limitation of

Erameh and Odoh (2021): a single-tenant system built for one establishment, with no mechanism by which hundreds of unrelated businesses could share the deployment securely.

Reviewing these systems side by side with the commercial platforms makes the market position clear. At one end, sophisticated multi-tenant SaaS platforms exist but are priced and configured for foreign markets. At the other, locally built systems fit the Nigerian context but are architecturally incapable of being offered cheaply to many businesses at once. The space between them a locally appropriate system with a genuinely multi-tenant, independently scalable architecture is unoccupied in the published literature.

2.6 The Nigerian SME Context and Technology Adoption

Any review of the systems literature for this project would be incomplete without the literature on the environment in which the system is meant to be used. The economic importance of small businesses in Nigeria is well documented. The collaborative MSME survey conducted by the Small and Medium Enterprises Development Agency of Nigeria and the National Bureau of Statistics reported that micro, small and medium enterprises account for the overwhelming majority of business entities in the country and contribute close to half of national GDP (SMEDAN & NBS, 2013). The same body of reporting has repeatedly identified limited access to appropriate and affordable business technology as a constraint on the sector's productivity.

Research on ICT adoption among Nigerian SMEs helps explain why the availability of commercial software has not by itself closed the gap. Afolayan et al. (2015), studying information technology usage among SMEs in a developing-economy context, found that ICT adoption among Nigerian SMEs was more advanced than might be expected, but that its further development was hampered by poor infrastructure and by costs that weigh far more heavily on small firms than on large ones. Their conclusion carries a direct implication for software design: the binding constraint is not willingness to adopt but the fit between the cost and infrastructure demands of the technology and the resources of the business. Apulu (2012), in a doctoral study of ICT adoption by SMEs in developing countries with Nigeria as the case study, reached a compatible conclusion, identifying inadequate infrastructure, the high cost of ICT relative to firm resources, and limited technical skills as the principal barriers, and arguing that adoption

strategies must be built around those realities rather than around the assumptions of developed markets.

This strand of literature does three things for the present project. It confirms that the problem addressed is real and documented rather than assumed. It identifies the specific barriers cost, infrastructure, and fit that a solution must respect, which is why the platform's architecture is organised around minimising per-tenant cost and why it is delivered through a browser without local installation. And it explains why simply importing the architecture of a Western SaaS product is not enough: the literature on multi-tenancy and microservices establishes how to build such a platform, while the Nigerian adoption literature establishes for whom and under what constraints it must be built. This project sits at the meeting point of the two.

2.7 Summary of Reviewed Literature

Table 2.1 summarises the twelve studies and reports that have most directly informed the design of this project, recording each work's focus, its methodology, its principal findings, and its limitations as identified by its authors or evident from its design.

Table 2.1: Summary of reviewed literature

Study	Focus	Methodology	Key Findings	Limitations
Chong and Carraro (2006)	SaaS architecture and maturity levels	Industry analysis and design guidance	Four-level SaaS maturity model; multi-tenancy and configurability identified as the sources of SaaS economies of scale	Practitioner guidance without empirical performance evaluation
Chong, Carraro and Wolter (2006)	Multi-tenant data architecture	Industry analysis of data-isolation options	Three data models (separate databases; shared database, separate schemas; shared schema) characterised as an isolation-versus-cost trade-off	Prescriptive; not validated against measured deployments
Guo et al. (2007)	Native multi-tenant application development	Requirements analysis and framework proposal	Enumerated the core requirements of native multi-tenancy: tenant identification, security isolation, customisation, quality-of-service management	Conceptual framework; not validated on a large deployed system
Aulbach et al. (2008)	Shared-schema multi-tenant databases	Experimental evaluation of	Generic mapping techniques degrade performance; chunk	Techniques tuned for extreme consolidation;

Study	Focus	Methodology	Key Findings	Limitations
		schema-mapping techniques	folding preserves it under shared-schema consolidation	complexity disproportionate for small deployments
Mietzner et al. (2009)	Tenant-specific customisation of SaaS	Variability modelling with deployment automation	Explicit variability models can drive automated, tenant-specific configuration and deployment	Model-level work with limited large-scale empirical validation
Bezemer et al. (2010)	Converting a single-tenant system to multi-tenancy	Industrial experience report with source-code analysis	Tenant identification, data isolation, and performance management are the recurring problem areas; retrofitting tenant-awareness is costly	Findings drawn from one legacy codebase
Apulu (2012)	ICT adoption by SMEs in developing countries	Doctoral case study of Nigerian SMEs	Infrastructure inadequacy, high relative cost, and limited skills are the principal adoption barriers	Qualitative study; findings directional rather than statistically general
Afolayan et al. (2015)	IT usage in Nigerian SMEs	Survey-based study in a developing-economy context	ICT adoption is advancing but remains constrained by poor infrastructure and cost burdens	Broad ICT focus; not specific to inventory or SaaS tools
Kabbedijk et al. (2015)	Definition and state of multi-tenancy research	Systematic mapping study of 761 papers and 371 industry blogs	Synthesised an agreed definition; found the field dominated by solution proposals with few industrial experience reports	Mapping study; assesses the literature rather than any system
Tizzei et al. (2017)	Microservices and product-line engineering for multi-tenant SaaS	Empirical tracking of an industrial system over time	Average of 62 percent of code reused across tenant deployments	Evidence from a single industrial system
Batista et al. (2022)	Migrating a monolith to multi-tenant microservices	Industrial migration case study with scalability evaluation	Improved scalability, easier management, and smoother third-party integration	Evaluation confined to one company's migration context
Erameh and Odoh (2021)	Web-based inventory control for a Nigerian SME	Design and implementation using the MERN stack	Role-based web inventory system with barcode tracking met the case business's needs	Single-organisation monolith; no multi-tenancy, no load evaluation
Chukwumuanya et al. (2024)	Web-based inventory system with demand forecasting	Implementation plus ARMA time-series modelling	ARMA(1,1) forecasting and cloud deployment improved inventory control in the case unit	Single-establishment system; not designed for shared,

Study	Focus	Methodology	Key Findings	Limitations
				multi-tenant operation
Batista et al. (2024)	Asynchronous workloads in multi-tenant microservices	Industrial study of workload management	Message-queue-based coordination absorbs per-tenant workload surges without degrading other tenants	Enterprise-scale context; parameters differ from small-business deployments

2.8 Identified Gap in the Literature

Read together, the reviewed bodies of literature leave a clearly bounded gap, and it is the gap this project occupies.

The architectural literature is mature. Multi-tenancy has a settled definition and a documented set of engineering requirements (Kabbedijk et al., 2015; Guo et al., 2007). The data-isolation options have been catalogued and their trade-offs analysed (Chong et al., 2006; Aulbach et al., 2008). The conversion pitfalls are documented (Bezemer et al., 2010), and the combination of microservices with multi-tenancy has been validated in industrial settings (Tizzei et al., 2017; Batista et al., 2022; Batista et al., 2024). Yet this entire body of work shares one characteristic: its evidence comes from large organisations, enterprise vendors, and developed-market infrastructure. The studies answer the question "can these architectures deliver scale, isolation, and reuse?" with a confident yes. They do not address the question that matters here can these architectures deliver those properties at a per-tenant cost low enough, and with infrastructure demands modest enough, to serve a small retailer in Osun State.

The locally grounded literature answers the complementary question. The Nigerian studies of inventory systems demonstrate working, contextually appropriate implementations (Erameh & Odoh, 2021; Chukwumuanya et al., 2024), and the adoption literature documents exactly what prevents small businesses from reaching the commercial alternatives (Afolayan et al., 2015; Apulu, 2012; SMEDAN & NBS, 2013). But the local implementations are single-tenant monoliths whose architecture forbids the one thing that would make them affordable at scale shared, secure, multi-customer operation.

Between these two literatures lies the unoccupied space: no published work reviewed here reports the design, implementation, and evaluation of a multi-tenant, microservices-based

inventory platform aimed at the affordability and infrastructure constraints of Nigerian SMEs. This project addresses that gap by building such a platform and evaluating it against three explicit criteria: response time under load, scalability as the tenant population grows, and the integrity of data isolation between tenants so that the question is answered with measurements rather than assertions.

2.9 Chapter Summary

This chapter has reviewed the literature on which the project stands. The conceptual review established the meaning of SaaS and the centrality of multi-tenancy to its economics, surveyed the three standard approaches to multi-tenant data storage and their trade-offs, traced microservices from their service-oriented roots, and examined the state of computerised inventory management for small businesses. The theoretical framework fixed the two positions that organise the design: service-oriented principles for the system's decomposition, and the SaaS maturity model of Chong and Carraro (2006) for its tenancy and scaling ambitions, with the platform targeting the model's fourth, fully scalable level. The empirical review found the multi-tenant microservices combination validated in industrial settings, the data-layer trade-offs well characterised, and the principal failure mode retrofitting tenant-awareness avoidable in a greenfield build. The review of existing systems found commercial SaaS inventory platforms economically out of reach for Nigerian SMEs and local research implementations architecturally unable to serve many businesses at once. The Nigerian adoption literature confirmed that cost, infrastructure, and fit not unwillingness are the barriers such a platform must respect. The next chapter describes the methodology through which these findings are turned into a working system: the research approach and data collection, the detailed system architecture, the choice of tools, and the development model followed during implementation.

References

Afolayan, A., Plant, E., White, G. R. T., Jones, P., & Beynon-Davies, P. (2015). Information technology usage in SMEs in a developing economy. *Strategic Change, 24*(5), 483–498.

Apulu, I. (2012). *Developing a framework for successful adoption and effective utilisation of ICT by SMEs in developing countries: A case study of Nigeria* [Doctoral dissertation,

University of Wolverhampton]. University of Wolverhampton Intellectual Repository and E-Theses.

Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., & Zaharia, M. (2010). A view of cloud computing. *Communications of the ACM, 53*(4), 50–58.

Aulbach, S., Grust, T., Jacobs, D., Kemper, A., & Rittinger, J. (2008). Multi-tenant databases for software as a service: Schema-mapping techniques. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data* (pp. 1195–1206). ACM.

Batista, C., Proença, B., Cavalcante, E., Batista, T., Morais, F., & Medeiros, H. (2022). Towards a multi-tenant microservice architecture: An industrial experience. In *Proceedings of the 46th IEEE Annual Computers, Software, and Applications Conference* (pp. 553–562). IEEE.

Batista, C., Cavalcante, W. B. R., Proença, B., Batista, T., & Morais, F. (2024). Managing asynchronous workloads in a multi-tenant microservice enterprise environment. *Software: Practice and Experience, 54*(2), 334–359.

Bezemer, C.-P., Zaidman, A., Platzbeecker, B., Hurkmans, T., & Hart, A. (2010). Enabling multi-tenancy: An industrial experience report. In *Proceedings of the 2010 IEEE International Conference on Software Maintenance* (pp. 1–8). IEEE.

Chong, F., & Carraro, G. (2006). *Architecture strategies for catching the long tail*. MSDN Library, Microsoft Corporation.

Chong, F., Carraro, G., & Wolter, R. (2006). *Multi-tenant data architecture*. MSDN Library, Microsoft Corporation.

Chukwumuanya, O. E., Onwurah, U. O., & Ihueze, C. C. (2024). Development of a web-based inventory management system for small businesses. *International Journal of Industrial and Production Engineering, 2*(2), 53–67.

Dragoni, N., Giallorenzo, S., Lafuente, A. L., Mazzara, M., Montesi, F., Mustafin, R., & Safina, L. (2017). Microservices: Yesterday, today, and tomorrow. In M. Mazzara & B. Meyer (Eds.), *Present and ulterior software engineering* (pp. 195–216). Springer.

Erameh, K. B., & Odoh, B. I. (2021). Design and implementation of a web-based inventory control system using a small medium enterprise (SME) as a case study. *NIPES Journal of Science and Technology Research, 3*(3), 211–219.

Guo, C. J., Sun, W., Huang, Y., Wang, Z. H., & Gao, B. (2007). A framework for native multi-tenancy application development and management. In *Proceedings of the 9th IEEE International Conference on E-Commerce Technology* (pp. 551–558). IEEE.

Kabbedijk, J., Bezemer, C.-P., Jansen, S., & Zaidman, A. (2015). Defining multi-tenancy: A systematic mapping study on the academic and the industrial perspective. *Journal of Systems and Software, 100*, 139–148.

Kavis, M. J. (2014). *Architecting the cloud: Design decisions for cloud computing service models (SaaS, PaaS, and IaaS)*. John Wiley & Sons.

Mietzner, R., Metzger, A., Leymann, F., & Pohl, K. (2009). Variability modeling to support customization and deployment of multi-tenant-aware software as a service applications. In *Proceedings of the ICSE Workshop on Principles of Engineering Service-Oriented Systems* (pp. 18–25). IEEE Computer Society.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly Media.

Small and Medium Enterprises Development Agency of Nigeria, & National Bureau of Statistics. (2013). *SMEDAN and National Bureau of Statistics collaborative survey: Selected findings (MSME Survey 2013)*. SMEDAN.

Tizzei, L. P., Nery, M., Segura, V. C. V. B., & Cerqueira, R. F. G. (2017). Using microservices and software product line engineering to support reuse of evolving multi-tenant SaaS. In *Proceedings of the 21st International Systems and Software Product Line Conference – Volume A* (pp. 205–214). ACM.